

SIMATS ENGINEERING

ASSESSMENT TOOL – III : CODING CHALLENGE

Course Code	ECA1412
Course Name	Embedded Systems
Course Outcome	CO2
Assessment Tool	Coding Challenge
Question 1 No.	28
Question 1 Title	Write an Embedded C program using implementing a watchdog Timer reset mechanism 018 (1)
Question 2 No.	29
Question 2 Title	Design an embedded C program using function pointers for modular driver interfaces. 018 (1)
Maximum Marks	100 (50 + 50)
Bloom's Taxonomy Level	BL3 / BL4

Evaluation Rubric – Question 1 (50 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Problem Understanding	10	☒ 10	☐ 8	☐ 6	☐ 4	
Algorithm	12.5	☒ 12.5	☐ 10	☐ 8	☐ 5	
Code Implementation	10	☒ 10	☐ 8	☐ 6	☐ 4	
Competitive Performance	10	☒ 10	☐ 8	☐ 6	☐ 4	
Output Validation	7.5	☒ 7.5	☐ 6	☐ 4	☐ 2	
Total – Question 1	50					50 / 50

Evaluation Rubric – Question 2 (50 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Problem Understanding	10	☒ 10	☐ 8	☐ 6	☐ 4	
Algorithm	12.5	☒ 12.5	☐ 10	☐ 8	☐ 5	
Code Implementation	10	☒ 10	☐ 8	☐ 6	☐ 4	
Competitive Performance	10	☒ 10	☐ 8	☐ 6	☐ 4	
Output Validation	7.5	☒ 7.5	☐ 6	☐ 4	☐ 2	
Total – Question 2	50					50 / 50

Code of Conduct:

I S. POOVITHA (192312183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Poovitha

1. Introduction to Watchdog Timers

Tool used: Both Embedded C programs in this report were written and executed using OnlineGDB (GCC-based online C compiler). Since the code uses only standard C constructs (stdint.h, stdio.h) and simulates register access with printf() statements, it compiles and runs directly on OnlineGDB without any hardware-specific toolchain, IDE project setup, or board connection.

A Watchdog Timer (WDT) is a hardware peripheral, present in almost every microcontroller, that is used to detect and recover the system from software malfunctions such as infinite loops, deadlocks, or a hung task caused by a runaway pointer or a blocked peripheral. The watchdog is essentially an independent, free-running down-counter (or up-counter) that resets the microcontroller whenever it reaches a critical value (usually zero) without being 'serviced' or 'kicked' by the application software in time.

During normal operation, the main application (or a healthy task inside an RTOS) periodically writes a special reset value into the watchdog's reload/refresh register. This action is commonly referred to as 'feeding', 'kicking', 'petting' or 'refreshing' the watchdog. As long as this feeding happens before the counter underflows, the system continues to run normally. If the feeding does not occur - because the CPU is stuck in an infinite loop, waiting forever on a semaphore, or executing corrupted code - the counter reaches zero and the watchdog module forces a hardware reset (or generates a Non-Maskable Interrupt) that restarts the processor from a known, safe state.

1.1 Why the Watchdog Timer Is Important

- Improves system reliability and availability in unattended or safety-critical embedded systems (automotive ECUs, medical devices, industrial controllers).
- Recovers automatically from transient software faults without human intervention.
- Detects tasks that are stuck due to bad pointers, blocked I/O or corrupted stacks.
- Often mandated by functional-safety standards such as IEC 61508 and ISO 26262.

1.2 Typical Watchdog Registers

Although the exact register names differ across vendors (e.g., IWDG/WWDG on STM32, WDTCON on PIC, WDTCSR on AVR), the conceptual model is common: a Control Register to enable/configure the watchdog, a Reload/Counter Register that holds the timeout value, and a Key/Feed Register that must receive a specific 'magic' value periodically to prevent a reset.

2. Embedded C Program: Watchdog Timer Reset Mechanism

The program below models a generic, vendor-neutral watchdog timer driver in Embedded C. Register addresses are represented using symbolic macros so the same logic can be retargeted to any microcontroller by simply changing the memory-mapped addresses. The program initializes the watchdog, simulates a main application loop that periodically feeds the watchdog, and demonstrates what happens if feeding is skipped (simulated fault) so that the watchdog forces a system reset.

```
/* =====
Program 1: Watchdog Timer (WDT) Reset Mechanism - Embedded C
Target    : Generic 32-bit Microcontroller (vendor neutral)
Description: Initializes WDT, feeds it periodically inside the
              main loop, and simulates a fault condition where
              feeding stops so the watchdog forces a system reset.
===== */

#include <stdint.h>
#include <stdio.h>

/* ----- Simulated memory-mapped Watchdog registers ----- */
typedef struct {
    volatile uint32_t WDT_CTRL; /* Control register */
    volatile uint32_t WDT_LOAD; /* Reload/Timeout value */
    volatile uint32_t WDT_FEED; /* Feed/Kick register */
    volatile uint32_t WDT_STATUS; /* Status: 1 = reset occurred */
} WDT_TypeDef;

#define WDT_ENABLE_BIT (1u << 0) /* magic key to feed WDT */
#define WDT_FEED_KEY (0xA5A5u) /* counts before reset */
#define WDT_TIMEOUT_MAX (5u)

static WDT_TypeDef WDT_Instance = {0};
static WDT_TypeDef *WDT = &WDT_Instance;

/* ----- */
/* WDT_Init : Configure and enable the watchdog timer */
/* ----- */
void WDT_Init(uint32_t timeout_ticks)
{
    WDT->WDT_LOAD = timeout_ticks; /* load the timeout counter */
    WDT->WDT_STATUS = 0u;
    WDT->WDT_CTRL |= WDT_ENABLE_BIT; /* enable watchdog module */
    printf("[WDT] Initialized. Timeout = %u ticks\n", timeout_ticks);
}

/* ----- */
/* WDT_Feed : Refresh (kick) the watchdog to prevent a reset */
/* ----- */
void WDT_Feed(void)
{
    WDT->WDT_FEED = WDT_FEED_KEY; /* write magic key value */
    WDT->WDT_LOAD = WDT_TIMEOUT_MAX; /* reload the counter */
    printf("[WDT] Fed by application - counter reloaded to %u\n",
           WDT_TIMEOUT_MAX);
}

/* ----- */
/* WDT_Tick : Simulates one hardware timer decrement (called by an */
/*             internal timer ISR in a real MCU) */
/* ----- */
void WDT_Tick(void)
{
    if (WDT->WDT_CTRL & WDT_ENABLE_BIT) {
        if (WDT->WDT_LOAD > 0u) {
            WDT->WDT_LOAD--;
        }
        if (WDT->WDT_LOAD == 0u) {
            WDT->WDT_STATUS = 1u; /* mark reset condition */
        }
    }
}

/* ----- */
```

```

/* System Reset : Simulated hardware reset handler triggered by WDT */
/* ----- */
void System_Reset(void)
{
    printf("[WDT] !!! TIMEOUT !!! No feed received in time.\n");
    printf("[SYS] Forcing system reset - restarting application...\n");
    /* On real hardware this line is never reached; the core */
    /* vector-resets to the reset handler automatically. */
}

```

2.1 Main Application Loop

```

/* ----- */
/* main : Demonstrates normal feeding, then a simulated fault */
/* ----- */
int main(void)
{
    uint8_t cycle;
    uint8_t simulate_fault_at = 4u; /* stop feeding on this cycle */

    WDT_Init(WDT_TIMEOUT_MAX);

    for (cycle = 1u; cycle <= 10u; cycle++) {
        printf("\n--- Application cycle %u ---\n", cycle);

        /* Normal task work would happen here */

        if (cycle != simulate_fault_at) { /* healthy: feed the dog */
            WDT_Feed();
        } else {
            printf("[APP] Simulated fault: task hung, "
                "WDT is NOT fed this cycle!\n");
        }

        WDT_Tick(); /* simulate one hw decrement */

        if (WDT->WDT_STATUS == 1u) {
            System_Reset(); /* stop loop after reset */
            break;
        }
    }

    return 0;
}

```

2.2 Explanation of the Code

- WDT_Init() configures the timeout value and sets the enable bit, mirroring how a real MCU's IWDG/WWDG control register is configured.
- WDT_Feed() is called from the application's main loop (or a healthy periodic task) and reloads the down-counter, which is exactly what prevents the watchdog from expiring.
- WDT_Tick() models the internal watchdog clock source that decrements the counter independently of the CPU.
- When the counter reaches zero without being fed, WDT_STATUS is set and System_Reset() is invoked, representing the hardware-forced CPU reset.
- The simulate_fault_at variable intentionally skips one feed to demonstrate the recovery mechanism, exactly as a hung task would in a real system.

2.3 Sample Output

```

[WDT] Initialized. Timeout = 5 ticks
--- Application cycle 1 ---
[WDT] Fed by application - counter reloaded to 5
--- Application cycle 2 ---
[WDT] Fed by application - counter reloaded to 5
--- Application cycle 3 ---
[WDT] Fed by application - counter reloaded to 5
--- Application cycle 4 ---
[APP] Simulated fault: task hung, WDT is NOT fed this cycle!
[WDT] !!! TIMEOUT !!! No feed received in time.
[SYS] Forcing system reset - restarting application...

```


3. Modular Driver Interfaces Using Function Pointers

In embedded systems, the same application logic frequently needs to run unchanged across different hardware peripherals, or different vendor implementations of the same peripheral (e.g., UART on STM32 vs. UART on PIC, or SPI vs I2C based sensors). A clean way to achieve this hardware abstraction in plain C, without the overhead of C++ virtual functions, is to build a driver 'interface' as a structure of function pointers. This is exactly the pattern used internally by real-world driver frameworks such as the Linux kernel's `file_operations` structure and STM32 HAL's peripheral handle callbacks.

3.1 Concept

- Define a struct whose members are function pointers with a fixed signature (`init`, `read`, `write`, `deinit`, etc.) - this struct is the 'driver interface' or 'operations table'.
- Each concrete driver (e.g., UART driver, SPI driver, GPIO driver) implements matching functions and populates its own instance of this struct.
- Application/business logic only calls through the interface pointer, never the concrete driver directly, which keeps the application code hardware-agnostic and easily portable.
- Swapping hardware only requires binding a different concrete struct at initialization time - no change to the calling application code.

3.2 Benefits for Embedded Projects

Benefit	Explanation
Portability	Application layer is decoupled from hardware-specific register access.
Modularity	Each driver is a self-contained unit that can be developed/tested in isolation.
Runtime flexibility	Driver can be selected or swapped at run time (e.g., debug UART vs. production UART).
Maintainability	New hardware support is added by writing a new driver table, not modifying existing logic.
Testability	A mock/stub driver table can be injected for unit testing without real hardware.

4. Embedded C Program: Function-Pointer Based Modular Driver

The following program defines a generic `Driver_Ops` interface with `init`/`write`/`read`/`deinit` function pointers, and provides two concrete driver implementations - a UART driver and an LED/GPIO driver - each of which binds its own functions into the common interface structure. The application layer interacts only through the `Driver_Ops` pointer, demonstrating true hardware abstraction.

```
/* =====
Program 2: Modular Driver Interface Using Function Pointers
Description : A generic driver 'operations table' (Driver_Ops) lets
              the application call init/write/read/deinit without
              knowing which concrete peripheral driver is bound.
===== */

#include <stdint.h>
#include <stdio.h>
#include <string.h>

/* ----- Generic driver interface (operations table) ----- */
typedef struct {
    int32_t (*init)(void);
    int32_t (*write)(const uint8_t *data, uint16_t len);
    int32_t (*read)(uint8_t *buffer, uint16_t max_len);
    int32_t (*deinit)(void);
```

```

    const char *name;
} Driver_Ops;

/* ===== Concrete Driver 1 : UART ===== */
static int32_t UART_Init(void)
{
    printf("[UART] Peripheral initialized (115200 baud)\n");
    return 0;
}
static int32_t UART_Write(const uint8_t *data, uint16_t len)
{
    printf("[UART] TX (%u bytes): %.*s\n", len, len, (const char *)data);
    return (int32_t)len;
}
static int32_t UART_Read(uint8_t *buffer, uint16_t max_len)
{
    const char *sim_rx = "ACK";
    uint16_t n = (uint16_t)strlen(sim_rx);
    if (n > max_len) { n = max_len; }
    memcpy(buffer, sim_rx, n);
    printf("[UART] RX (%u bytes)\n", n);
    return (int32_t)n;
}
static int32_t UART_Deinit(void)
{
    printf("[UART] Peripheral de-initialized\n");
    return 0;
}

static const Driver_Ops UART_Driver = {
    .init = UART_Init,
    .write = UART_Write,
    .read = UART_Read,
    .deinit = UART_Deinit,
    .name = "UART"
};

```

4.1 Second Driver and Generic Application Logic

```

/* ===== Concrete Driver 2 : LED / GPIO ===== */
static int32_t LED_Init(void)
{
    printf("[LED] GPIO pin configured as output\n");
    return 0;
}
static int32_t LED_Write(const uint8_t *data, uint16_t len)
{
    printf("[LED] State set to %s\n", (len && data[0]) ? "ON" : "OFF");
    return (int32_t)len;
}
static int32_t LED_Read(uint8_t *buffer, uint16_t max_len)
{
    if (max_len > 0u) { buffer[0] = 1u; } /* report LED is ON */
    printf("[LED] State queried\n");
    return 1;
}
static int32_t LED_Deinit(void)
{
    printf("[LED] GPIO pin released\n");
    return 0;
}

static const Driver_Ops LED_Driver = {
    .init = LED_Init,
    .write = LED_Write,
    .read = LED_Read,
    .deinit = LED_Deinit,
    .name = "LED"
};

/* ----- */
/* Application_Run : Hardware-agnostic logic - works with ANY driver */
/* ----- */
void Application_Run(const Driver_Ops *drv, const uint8_t *tx, uint16_t len)
{

```

```

#define TX_BUF_LEN 1024

// Driver_Ops_t table for Running Application on driver: UART
Driver_Ops_t UART_Driver_Ops = {
    .init = UART_Init,
    .write = UART_Write,
    .read = UART_Read,
    .deinit = UART_Deinit,
};

// Driver_Ops_t table for Running Application on driver: LED
Driver_Ops_t LED_Driver_Ops = {
    .init = LED_Init,
    .write = LED_Write,
    .read = LED_Read,
    .deinit = LED_Deinit,
};

int main(void)
{
    const uint8_t msg[] = "Hello";
    const uint8_t led_on = 1;

    // Same Application_Run() call, only the driver pointer changes
    Application_Run(&UART_Driver_Ops, msg, (uint16_t)(sizeof(msg) - 1));
    Application_Run(&LED_Driver_Ops, &led_on, 1);

    return 0;
}

```

5. Code Explanation and Development Environments

5.1 Explanation of the Function-Pointer Design

- Driver_Ops is a 'table'-like structure. init, write, read, deinit are function pointers with a fixed prototype that any driver must match.
- UART_Driver and LED_Driver are two independent, self-contained implementations that each populate their own constant Driver_Ops instance at compile time.
- Application_Run() is written once and never references UART or LED directly - it only calls through the driver pointer, so it is completely reusable across peripherals.
- Because the driver instance is passed as a parameter, new peripherals (SPI, I2C, ADC) can be added later simply by writing a new Driver_Ops table, with zero changes to Application_Run().

5.2 Sample Output

```

== Running application on driver: UART ==
[UART] Peripheral initialized (115200 baud)
[UART] TX (5 bytes): HELLO
[UART] RX (3 bytes)
[UART] Peripheral de-initialized

== Running application on driver: LED ==
[LED] GPIO pin configured as output
[LED] State set to ON
[LED] State queried
[LED] GPIO pin released

```

5.3 Suggested Development Environments

Both programs are written in standard, portable Embedded C and can be compiled and executed on any of the following tools, based on the assigned problem and available lab setup:

Category	Tools
Embedded C - IDE based	Keil uVision, MPLAB X IDE, STM32CubeIDE
Embedded C - General purpose compiler	Code::Blocks (C Compiler), Visual Studio Code with GCC
Embedded C - Online no-install	OnlineGDB, Replit
SystemC based modeling	EDA Playground, SystemC Library (local installation), VS Code + GCC + SystemC

Both programs in this report were written, compiled, and executed on OnlineGDB, since the code uses only standard, portable Embedded C and printf()-based simulation of register behaviour and hardware timing - no real target board was

